

Abstract

In this project, we compare four priority queue implementations using data collected from a LinearBaseline, BinaryHeapInVector, BinomialQueue, and a quadratic Oracle. Each implementation was evaluated using the batch-then-drain profile with traces generated from a single fixed seed to provide consistent randomized data. The output of the BinaryHeapInVector and BinomialQueue was validated against the Oracle, and all implementations were timed using a harness that measures only the replay loop.

We found that BinaryHeapInVector outperformed BinomialQueues as well as the Oracle past, roughly $N = 2^{14}$ items.

Question and Hypotheses

Question:

How do four priority-queue implementations behave using an offline batch-then-drain profile where all inserts occur at once and the rest of the trace consists entirely of repeated `findMin / deleteMin` operations?

Hypothesis:

Because the drain is a delete-heavy operation, we expect implementations with more favorable `deleteMin` costs to perform better at larger N . The binary heap should outperform the binomial queue once N grows large. The LinearBaseline should stay low because it represents a constant "floor" factor, $O(1)$ time. The Oracle is expected to diverge quickly because of its $O(N^2)$ runtime.

Method

The priority-queue API for this experiment uses the same four operations: `insert`, `findMin`, `deleteMin`, and `extractMin` (other operations exist but were not used). Items `(key, id)` were compared as follows: key ascending, then id ascending.

Traces were generated using the batch-then-drain generator and a single fixed seed `{23}`. The workload first inserted all items, then repeatedly called `findMin` followed by `deleteMin` to drain the structure. Duplicates were minimized. All traces were validated using the quadratic Oracle which ensured correctness.

The harness measured only the replay loop of the operations recorded in the trace file and reported the median from five trials for each `(N, seed)` pair. All four implementations were evaluated under identical conditions on our local machine, running minimal background apps to ensure a stable system and a constant power source to ensure stable power.

Datasets

The batch-then-drain profile produced 12 trace files. Each trace represented a single (N, seed) pair. Trace files live in `traces/batch_then_drain/`

Trace file format:

- `batch_then_drain_N_<N>_S_<seed>.trace`
 - `N_<N>` = number of elements
 - `S_<seed>` = random seed

The distribution for keys was established with `uniform_int_distribution(1, 2^20)`.

- N ranged from 13 and approximately 2^{10} ... 2^{20} except the Oracle, which was stopped at 2^{15} to avoid y-axis blowout and high runtimes.

A single fixed seed `{23}` via `std::mt19937` ensured that randomness was consistent and repeatable.

Dataset Summary

Parameter	Value
Profile	Batch-then-drain
Range	1 to 2^{20}
N values	13 & 2^{10} to 2^{20} (Oracle $\leq 2^{15}$)
Seeds	Single Fixed

CSV Dataset Summary

Column	Meaning
impl	PQ implementation tested
profile	batch_then_drain
trace_path	path to source <code>.trace</code> file
N	number of inserts and number of extracts
seed	RNG seed used
elapsed_ms	total runtime for replay
ops_total	total operations in trace (= 2N)
inserts	always N

Column	Meaning
find/deleteMin	N

Results for Batch-Then-Drain

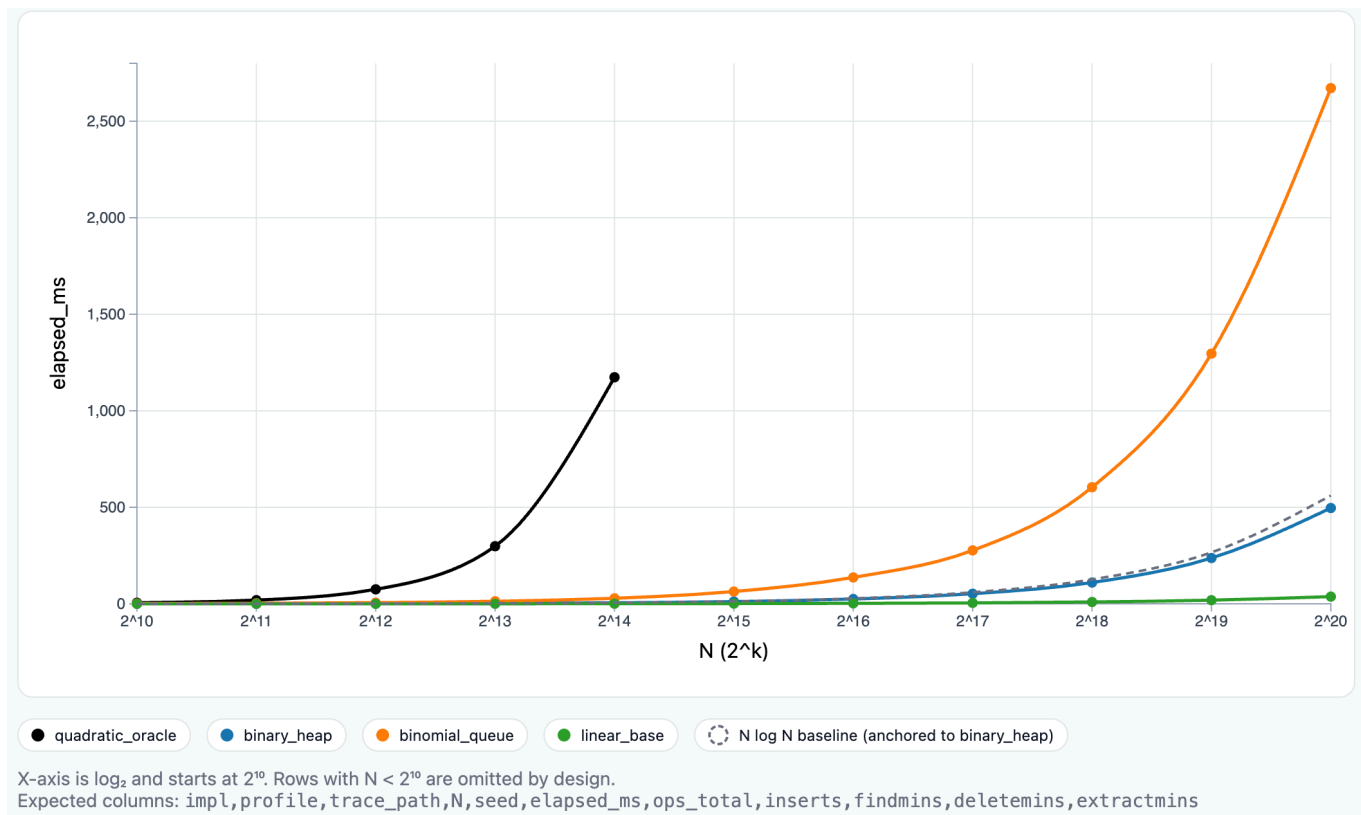


Figure 1 shows time vs. N for the batch-then-drain profile.

The binary heap curve stays lower than the binomial queue beyond $N = 2^{14}$ and remains consistently below both the binomial queue and the Oracle at larger size N's. As expected, the LinearBaseline remains near the bottom of the plot. The Oracle sharply increases beginning near $N = 2^{12}$ and would dominate the y-axis at larger N, which is why it was capped at 2^{15} . Overall, the picture matches our hypotheses.

Discussion

Because batch-then-drain is dominated by `deleteMin`, PQs with cheaper deletion costs perform better at larger N. For small N, constant operations are more dominant and all curves stay low on the graph and relatively close together. As N increases, the true behavior becomes apparent and is as follows:

- **BinaryHeapInVector** performs best because `deleteMin` is $O(\log N)$ with a low constant work factor.
- **BinomialQueue** performs well, but its deletion cost and structural puts it at a disadvantage as N grows.

- **LinearBaseline** remains low, as intended, since it acts as an almost constant time lower bound.
- **Oracle** rises the fastest, which is expected of a quadratic implementation, and serves to validate correctness.

Once the workload becomes mostly deletions, the data structure with the most efficient deleteMin, BinaryHeapInVector, is the clear winner.

Takeaway

BinaryHeapInVector is the best implementation for large, delete-heavy workloads. BinomialQueue performs well and comparably at smaller N because constant factors keep its runtime in check, but it loses that edge after $N = 2^{14}$. The LinearBaseline provides a constant "floor" factor reference. The Oracle is useful for correctness and shows expected quadratic growth. It is obviously not competitive beyond small N sizes.

Artifacts

All code, traces, CSV files, and figure scripts are included in the repository.

- Traces: `traces/batch_then_drain`
- Timing CSV: `csvs/batch_then_drain.csv`
- Figure HTML: `charts/pq_multi_impl_anchor_heap_tooltips_btd.html`
Reproduction instructions for Figure 1 are included in the README (section 3.4).